AI CODING

There are two ways to code your sources with AI, and for most people the easier one is to ask.

With **Edit my data with AI** switched on, tell MapCat what you want in plain words and it does the whole job: it picks the settings, starts the run, and confirms with you before anything changes. You do not need to know which controls exist. If the switch is off when you ask, MapCat offers it to you in the chat as a single button. The end of this section lists what you can ask for.

The **AI Coding panel** in the left pane is the manual route, and it is the one to use when you already know which settings you want, or when you want to change something MapCat has set. It is worth reading either way, because what MapCat sets is what these controls end up holding.

Quick start: If you have roughly 5–100 pages of text, you can usually **just run everything** and get decent results. Press **One-click coding** and confirm the short set-up modal, then let it run. You can then go back and adjust the coding (edit links, tweak prompts, re-run specific steps) if you want. For longer texts or high-stakes coding, work incrementally: use the **source limit** in Auto-code (1, 5, 20%, 50%, 100%) and the **Links limit** in Recode to process a sample first, check quality, then scale up.
AI is switched on and off in the Account panel with the **"AI options switched on and active"** switch. If you choose an AI workflow when you sign up, it is turned on for you; otherwise you can turn it on there at any time. When AI is on, the **AI Coding panel** appears inline at the top of the **Create links** tab, ready to use.

AI usage consumes **credits** (see Responses Panel). **One credit is a fixed unit of AI spend worth about half a US cent**, so 100 credits is roughly 50 cents of usage. There is no fixed credits-per-page or credits-per-token rate: the app works out the real cost of each AI call from its token usage and the model's price, then converts that to credits at half a cent each. So the credits a task uses depend entirely on the model you choose, the length of your codebook, and how many passes you run. Credits renew monthly and do not roll over.

As a rough ballpark for a single Auto-code pass over 30 pages of text: around 30 to 40 credits on the default **Gemini 3.5 Flash**, about half that on **Gemini 3.6 Flash**, and about a sixth of it on **Gemini 3.1 Flash-Lite**, the cheapest model in the list. **Gemini 3.1 Pro Preview** costs about a third more per token than 3.5 Flash and usually writes more, so budget above that. One-click coding costs more than one Auto-code pass, because it also runs Revise codebook, Recode and the join-islands pass. Running extra prompt sections or re-runs multiplies the cost. These figures are estimates and will change as model prices change; the Responses Panel shows what your own runs actually cost.

Users with dedicated AI plans receive a larger batch of AI credits each month; other users receive 100 free AI credits per month (the free credits do not stack with paid plans). The per-plan allowances are listed under AI credits.

The AI Workflow

The AI Coding panel sits at the top of the **Create links** tab, above the source text viewer. The Sources bar, the right-hand output tabs, and the **Create links** / **Filter links** / **Assess links** tabs all stay where they are; the panel does not take over the screen.

The panel is broken down into five straightforward sections (filtering is a separate step, see below):

1. **One-click coding**: Pipeline runner with a **set-up** modal first. Press **One-click coding** to choose **level of effort** (Flash vs Pro for the AI model slots), **Skip coded**, **Filter on finish**, and (if the project has links) whether to delete **every** link in the project or **only** links on the sources in scope, then confirm **Run**.
    - Pre-steps: clears Filter Links and turns the filter pipeline **on** before the modal.
    - **Scope**: One-click respects the Sources bar (empty bar = all sources, otherwise your current selection) and the sources % radio, exactly like the Auto-code button (no separate "code all sources" toggle). The modal states how many sources Auto-code will run on, including the % sample and when **Skip coded** removes already-coded sources (and **Run** is disabled if nothing is left).
    - **Auto-code prompt**: One-click coding uses the current Auto-code panel prompt and settings. The prompt is shown in the set-up modal before you run.
    - **Single source** in scope: only **Auto-code** runs (Revise codebook and Recode are skipped so labels are not merged across several AI passes). **Filter on finish** defaults off in that path but you can turn it on.
    - **Several sources** in scope: **Auto-code**, then **Revise codebook**, then **Recode**.
    - **Recode target suffix**: Choose blank (simpler — synthesised labels go straight into cause/effect) or e.g. _recoded (keeps raw labels, writes synthesised to temp columns so you can compare).
    - Per-step **Run** buttons still work on their own; the modal suppresses the extra confirms for the sequenced run after you confirm **Run**.
2. **Background**: Give the AI project context before coding. A status tick indicates whether enough background text is set.
3. **Auto-code**: This is where the AI reads your documents and extracts causal links.
    - You can choose to process a small sample first (e.g., `1` or `5` sources) to test your prompt, or process `100%` of them.
    - The "Skip coded" switch ensures you don't waste time and money re-processing documents that already have links.
    - Default model is **Gemini 3.5 Flash**.

4. **Revise codebook**: Once you have some causal links, the AI can review them and suggest a cleaner, more consistent list of factor labels (a "codebook"). The header tick shows whether the Recode codebook area currently contains suggestions.
    - Includes a **Target clusters** slider; see Target clusters.
    - Optional **Use automatic pre-clustering** switch (default OFF).
    - When pre-clustering is OFF, the AI tries to find the clusters directly from the factor list using the standard Revise codebook prompt. This prompt supports macro replacement: use `[number]` (or `[cluster_count]`) and the **effective** target cluster count is injected at run time (same as the slider logic below).
    - When pre-clustering is ON, the app first groups factor labels semantically using embeddings, then sends those clustered groups to the AI with a separate labelling prompt plus a **Representatives per cluster** slider (`8` to `20`, default `8`).
    - Pre-clustering is more systematic than asking the AI to find all clusters "in its head" from a long raw list. It reduces the black-box / WEIRD-data risk a bit, and may make it easier to preserve more unusual or divergent concepts instead of collapsing them into whatever the model finds most typical.
    - Default model is **Gemini 3.5 Flash**.
5. **Recode**: Apply the AI's suggested, cleaned-up labels back to your existing causal links. Paste the codebook (from Revise codebook or your own), add a recode instruction, and run.
    - In **Semantic** mode (formerly “AI factors”) the AI returns index mappings (row → codebook item) rather than full label text, reducing tokens and improving reliability; in **Causal** mode (formerly “AI links”) it returns a recoded cause and effect for each link.
    - **Semantic** and **Causal** each have their **own recode-instruction box, default and history** (separate prompt channels), and only the box for the selected mode is shown. Semantic default: *"For each raw label give me the number of the best-matching codebook item by meaning. Use 0 when no codebook item fits. Never invent labels."* Causal default: *"Recode each link to the codebook: pick the best-matching label for its cause and for its effect, using the link quote for context."*
    - **Skip recoded**: When on, only processes links that have at least one unrecoded label (cause or effect). Use this when recoding again to focus on remaining work.
    - **Links limit** (1, 5, 20%, 50%, 100%): When not 100%, a random sample of links is recoded. Non-sampled links keep their existing recoded values (or stay blank on first run).
    - The header progress bar is segmented: grey = empty recoded fields, orange = recoded equals original cause/effect, green = recoded non-empty and different.
    - Default model is **Gemini 3.5 Flash**. After coding, **filtering** happens in the adjacent **Filter links** tab (the normal Filter Links pipeline). When **Filter on finish** is **on** in the One-click set-up, completing the run applies these analysis filters to the pipeline: **Factor Frequency** (top `12`, counted by **citations**) → **Link Frequency** (top `30`, counted by **citations**). The global Label set controls which `cause`/`effect` columns Recode writes to (no separate “recode suffix” in this panel).

What the coding prompt decides

The conventions the AI follows when it turns text into links are defined by the **Auto-code prompt**, which you can read and edit in the Auto-code section (it is also shown in the One-click set-up modal before a run). That prompt is the single source of truth, so to see exactly how any case is handled, or to change it, read or edit the prompt rather than relying on a summary. In outline it covers:

- **Hierarchical labels**: factors can be labelled `General; specific` (two or more free levels), most general first, so the map can later be zoomed out by truncating at a semicolon. Whether to use them is a choice: it suits a large corpus with a multi-level scheme, and is left off for a small project or a fixed codebook. When it is on, the coder is also told what the top (most general) level should read like, for example a social-science concept, an actor name, or a codebook's general categories. You can ask MapCat to turn hierarchical labels on or off.
- **Actor focus** (default on): factors name who or what the cause and effect are about, with the actor as the subject and no passive voice, which suits maps that track who affects what. You can ask MapCat to turn this off for a concept or thematic map, and the coder then labels in the source's own words instead.
- **Opposites**: a leading `~` marks the reverse pole of a factor (for example `~Employment` for "less employment"), so both polarities share one node.
- **Failed influences**: a link aimed at an outcome that did not follow is still coded, with the **failure** column set to 1 and the arrow pointing at what the influence was for.
- **Every segment accounted for** (default on): the coder splits each source into short segments and must account for each one, finding the causal claims in it or recording that it has none, rather than stopping once it has found a few links. This raises recall on long chunks. To keep it reliable, chunks are processed at 16,000 characters even when you set a larger chunk size.

The prompt states the full rules and their interactions; this list is only an orientation.

One-click coding (AI)

- Sequencer for the AI pipeline, with one **set-up** modal (see step 1 under The AI Workflow).
- **Run** starts **Auto-code**; with **more than one** source in scope it continues with **Revise codebook** then **Recode**, stopping on the first non-successful stage. With **exactly one** source in scope, it **stops after Auto-code**.
- One-click uses the same panel prompts/settings as the separate step buttons, while skipping the extra per-step run confirmations after you confirm the one-click set-up modal.
- Existing links on the sources in scope are always replaced: they are deleted just before the run so the new coding does not duplicate them. Links on other sources are kept. The set-up modal shows the counts.
- **Recode target**: Use the global Label set below the Sources bar. Create a new suffix there first if you want Recode to fill `cause_suffix` / `effect_suffix` instead of only the default columns.
- **Join islands (final step, default on)**: after coding, one-click runs a join pass so the map comes out connected rather than left in many small islands. It merges near-duplicate factor labels and adds

cross-island links, each backed by a verbatim quote from the same source. These added links are the least certain part of the map, so review them if precision matters. The join stays within a single source; connecting across sources is Recode's job. If it fails it is skipped and the rest of the run still completes. The pass decides for itself when to stop: when the map has come out in one piece, when a round adds nothing, or when the island count stops falling (it allows one grace round, because a merge can enable a bridge in the round after it). It reports which of those ended it, so a run that converged early does not read as one that ran out of budget. Through MapCat you can set the join to **off** or to **merge only** (tidy and consolidate labels without adding any new links); leave the number of rounds alone, because a number you name overrides the stopping rule above. **merge only** suits reproducible coding or very long sources where no invented links are wanted. You can also name the model the join pass should use, separately from the coding model, for example "code with Flash then join islands with Pro".

Background (AI)

- Sets shared project context used by AI coding prompts.
- The status tick indicates whether enough background text is present.

Auto-code (AI)

- Runs AI coding across selected/all sources using your prompt and model.
- **Layout (top → bottom): Model**, **Skip coded**, **Add source prompt**, and **source limit** row; then the **Prompt sections** editor; then **Advanced** (chunk size, concurrency, temperature, thinking, etc.).
- Use source limit + skip coded options to test quickly and avoid rework.
- **Add source prompt** (switch): when ON, each source's optional *Source Prompt* (edit above the source text when viewing a source) is prepended to your main Auto-code prompt for that source. Saved per project in the browser. Use when sources need different context; skip when one background prompt in **Background** is enough.
- **Status line** under the settings shows progress, per-chunk detail, and stop — same behaviour as the former "Code with AI" card.
- **Prompt sections**: use **Add section** / **Remove** in the UI to split one saved prompt into reproducible iterations. Internally this is still stored as one prompt with `====` separator lines. Later sections see prior user/assistant turns. Only the last iteration's result is written to links; all iterations appear in Responses. This is best for workflows where coding is genuinely better in stages, such as first building the network, then adding columns like Time or Certainty, then running a checking pass.
- **Standard prompt builder**: the button above the sections opens a panel of tested prompt parts — the core coding rules plus optional extensions (hierarchical labels; in-vivo labels, which is actor focus turned off; opposites; despite; packages; a whole-network framing; and the sentiment, type and emotion columns), a prune pass, and your project codebook. The ticks are built from the prompt manifest rather than a fixed list, so a part added later appears here on its own. Tick what you want

and **Replace prompt**: the composed prompt lands in the section cards, fully editable, and saves to history as normal. Parts with prerequisites tick them automatically (despite needs opposites).

  - **Project codebook**: one of the ticks is your own codebook, the factor labels saved under Project Details > **Edit codebook**. Ticking it puts the whole list into the prompt, so the coding run works to your labels instead of inventing its own. The tick is greyed out until the project has a codebook, and it shows how many labels it will use. Two ways to use it:
    - **Guided** (the default): the coder uses one of your labels wherever one fits, and writes a new label in the same style where none does. Your list grows.
    - **Strict**: the list is closed. The coder reproduces a label as written, and does not code a claim it cannot express in your labels. Your list stays exactly as it is, and some claims in the text go uncoded, which is the point of choosing it.
  - On a project using hierarchical `General; specific` labels, the codebook is read as the list of general concepts, and the specific part after the semicolon still comes from the claim.
  - MapCat reaches the same thing from chat: it can read the codebook, add labels to it, and put it into the coding prompt in either mode. Asked that way it appends the codebook section to the prompt you already have, rather than replacing your prompt with a composed one.
  - This changes the prompt for the NEXT run. To bring links you have already coded onto the codebook, use **Recode** instead.
- **Per-section model**: each section after the first has its own model dropdown, defaulting to **Main model**. Use it to code with a cheap high-recall model and prune with a stronger one (for example a Flash model coding and Gemini 3.1 Pro Preview pruning), which testing showed removes most wrong links while keeping the good ones. All sections in one run must be from the same provider.

- **Rerun from here**: each prompt section has a small rerun button. Use it to continue a stable multi-section prompt without paying again for earlier successful stages, not as an open-ended chat workflow for coding maps. Section 1 reruns normally. Later sections reuse the latest successful earlier iteration history only when the earlier source text, prompt sections and chunk bounds are unchanged; otherwise the run fails loudly. You can add new sections under a successful run and rerun from the first new section.

- **Confirm** before a run shows model, chunking, word count, and cost estimate. **Stop** cancels after current chunk tasks finish.
- **Chunk size** (Advanced) defaults to **16K** characters. Larger chunks give the model more context per call and fewer islands to join afterwards; lower it only if a model struggles with long inputs. The every-segment-accounted-for pass caps the effective size at 16,000 characters even when you set a larger value.
- Timeouts scale by model and iteration count (cap ~540s total). **Concurrency** (1–5) is in Advanced; raise for speed, lower if you see 429/timeouts.
- **Which prompt a run used**: every confirmed run saves the prompt it sent and stamps that prompt's id on the run, so a map can be traced back to the wording that made it. Ask MapCat to list this project's coding prompts with the runs each one produced, and to put one back in the panel. It restores by id rather than by "the most recent", because the panel always shows the newest saved

prompt and a later run moves it, which is what used to make a prompt look as though it had reverted on its own.
- **The prompt a run sends is not quite the prompt you see**: the run appends the per-segment accounting instructions (see What the coding prompt decides) and, when it is on, each source's own Source Prompt. Ask MapCat to read out the effective coding prompt if you want the exact string that goes to the model.
- Tips on using the prompt history (same chrome as other prompt fields).
- Default model is **Gemini 3.5 Flash** (precise and EU-resident; in testing it produced very few wrong links at moderate volume). One-click coding inherits this default.

While a coding run is going

- Before a run starts, it shows how many sources are in scope, coloured by how much of the project that is: red for every source, amber for a large slice, grey for a small one, with the chunk count, chunk size and model underneath. A five-second countdown then runs, so you can press **Stop** to abort before any AI work starts if the scope looks wrong. One-click runs skip the countdown, because they confirm the scope earlier, but still show the header.
- While the run works, a progress panel shows the current chunk, an estimated finish time, and a **Stop run** button. The browser tab title also tracks progress, so you can watch it from another tab.
- Press **Continue in the background** to hide the panel and carry on working. The run keeps going on the server and tells you when it finishes; reopen the panel any time from the AI Coding panel. Server runs are safe to close the tab on; the panel says so once the run is server-side.
- Only one coding run happens at a time. While a run is live the **Run** buttons become a stop control and the other AI actions are greyed out, so you cannot start a second run over the same sources by accident.

Revise codebook (AI)

- Suggests a cleaner consolidated codebook from existing links.
- Use this after you have enough coded links for a representative sample.
- Header tick indicates whether the Recode codebook area currently has content.
- **Target clusters**: see Target clusters.
- Optional **Use automatic pre-clustering** switch (default OFF).
- With pre-clustering OFF, the AI clusters the factor list directly from the Revise codebook prompt. That prompt supports `[number]` / `[cluster_count]`.
- With pre-clustering ON, embeddings are used first to group labels semantically, then the AI only has to label those grouped clusters. This is a bit more systematic, less dependent on the AI doing all clustering internally as a black box, and may help preserve unusual or divergent concepts.
- Pre-clustering also adds a **Representatives per cluster** slider (8-20, default 8) and uses a separate labelling prompt.
- Default model is **Gemini 3.5 Flash**.

Target clusters (Revise codebook)

- The **Target clusters** control is a slider with **50 positions**. The **far left** is **Default**; moving right sets an explicit target of **2** through **50** clusters (one step per cluster count).
- **Default** (far left): the app derives a target count K from the number of **unique factor labels** n in the **current filtered pipeline** (same scope as Revise codebook): $K = \min(\lfloor n/3 \rfloor, 25)$ — at most **25**, or roughly **one label in three** as clusters, whichever is smaller.
- **Explicit** positions (not Default): the requested K is the number shown by the slider (**2–50**). If K is **greater than** n, the run uses n instead (you cannot have more clusters than distinct labels); the app may show a short notice when that cap applies.
- Pre-clustering, embedding clustering, and `[number]` / `[cluster_count]` in prompts all use this **effective** K.

Recode (AI)

- Applies your codebook back onto existing links, turning raw factor labels into cleaner synthesised ones.
- The **Recoding** radio buttons map onto the kinds of recoding (see Different kinds of coding and recoding). Overall the four run in order of **increasing cost and increasing quality**, left to right: Magnetic, then Semantic (formerly “AI factors”), then Causal (formerly “AI links”), then Hard. Pick the cheapest one that is good enough for your data. The names say what the model reads. Magnetic and Semantic both judge a label by what it means, one by vector and one by reading it. Causal judges the claim the link makes, which is why it needs the quote.
    - **Magnetic** (soft recoding): embedding similarity to codebook lines, the same magnet machinery as the pipeline's soft-recode path, with a similarity threshold. No AI call, and only as good as the embedding space.
    - **Semantic** (factors recoding, formerly “AI factors”): the model relabels each unique factor label to its best-matching codebook line, reading the label and nothing else. Cheap, because it costs one row per distinct label however many links carry it, and blind to what any of those links actually claimed. You can bring other factor columns into play (citation count, source count) when you write the instruction.
    - **Causal** (links recoding, formerly “AI links”): the model relabels each link from its cause, effect and the actual quote, so it judges by the causal claim rather than by the wording, almost like recoding from scratch. It can also send the same label two ways in two links where the quotes show it doing different work, which Semantic cannot do at all. Dearer, because cost follows the number of links. This is the default.
    - **Hard** (hard recoding): re-codes the source text from scratch against the codebook rather than relabelling existing links. Pick the sources (**Coded sources**, the ones already coded, or **All sources**), confirm, and it re-runs AI coding with the codebook as a closed list (the default instruction forbids inventing labels; your own instruction may relax that), replacing those sources' links. It uses the Auto-code model and chunk settings.

- The names refer to the mapping method, not "soft recoding" in the filter sense.
- **Watch the batch size**: with a large set of links and long quotes, a recode run is split into multiple calls, and each call can settle on slightly different labels. If you want a small fixed set of top-level labels, develop them first with Answers mode or the Cluster part of the Soft Recode filter, then recode against that codebook.
- **Recode target**: the global Label set. The default set writes the standard cause/effect labels; a named set writes that set's own cause and effect columns instead, leaving the default pair untouched.
- Supports sampled recoding and skip-recoded behavior (skip-recoded only applies when using a non-default label set).
- Header bar shows recode coverage mix across all cause/effect recoded fields.
- Default model is **Gemini 3.5 Flash**.

Filter links (AI)

- This is the same Filter Links workflow, run from the adjacent **Filter links** tab (not embedded in the AI Coding panel).
- Use it to refine/select links before reviewing outputs on the right.
- When **One-click coding** finishes with **Filter on finish** enabled, the app applies top-12 factor frequency (citations), then top-30 link frequency (citations) (no longer injects the deprecated Temporary Factor Labels filter).

Advanced Settings

Each section header is clickable and opens/collapses its settings panel. Section headers also include contextual **Help** buttons. The advanced sections are inline (not flyouts), and only one section is expanded at a time.

Inside advanced panels you can:

- Edit the exact **Prompt** the AI uses.
- View your prompt history and load previous prompts.
- Change the **AI Model** (e.g., switch to a "Pro" model for complex reasoning, or a "Flash" model for speed).
- Tweak technical settings like chunk size, concurrency, and temperature.

Coding by chat instead

Everything above can be asked for in words. With **Edit my data with AI** on and an AI plan, MapCat drives this panel for you.

- **Code my sources.** MapCat asks a couple of plain questions about how you want them coded, sets the controls, and starts the run. Ask for the plain coding pass and it runs **Auto-code** alone; ask for

the whole pipeline and it runs one-click coding, with revise, recode, join islands and the finishing filters behind it.

- **Set up before running.** Level of effort or a named model, chunk size, skip coded, filter on finish, which sources are in scope, hierarchical labels on or off, actor focus on or off, the recode mode, the recode instruction, and the codebook.
- **See what it chose.** Before a run, MapCat shows a default-collapsed **coding decisions** card listing the conventions it settled on, and the same decisions go into the run's logs, so a map can be explained afterwards rather than guessed at.
- **Afterwards.** Ask it to join up a fragmented map, suggest a revised codebook, recode the links to a codebook you agree in chat (which is how you code top-down to your own themes or theory of change), or delete the coding for a set of sources and start again.
- **Trace and restore prompts.** Ask which prompt a run used, what you have tried on this project, and to put a particular one back in the panel.

Every coding run confirms in the chat before it starts. That card names the sources in scope, so you see what is about to be coded before you agree to it.